Sistemas Robotizados y Expertos

Modelos de Inteligencia Artificial

Curso de Especialización en IA y Big Data

1. Fundamentos de Robótica y Hardware para IA

Este capítulo sienta las bases físicas y de hardware sobre las que se construirá todo el trabajo posterior. Antes de escribir una sola línea de código de IA, es imprescindible entender qué es un robot, cómo está construido y qué restricciones impone el mundo físico sobre los algoritmos que lo controlan.

1.1. Tipologías de Robots y Diseño Físico

En el ámbito de la Inteligencia Artificial y el Big Data, el robot se define como un sistema físico capaz de **percibir** su entorno (mediante sensores), **procesar** dicha información (mediante software y modelos de IA) y **actuar** sobre el mundo real (mediante motores y actuadores). A diferencia de un programa de software que solo existe en el mundo digital, el robot introduce restricciones adicionales críticas: debe operar en tiempo real, con energía limitada (batería), bajo peso y con capacidad de disipación de calor reducida.

El diseño físico del robot —su **morfología**— no es una decisión meramente estética; determina el **Espacio de Trabajo** (los puntos del espacio que puede alcanzar), su **Capacidad de Carga** máxima y la complejidad del algoritmo de control necesario. Para el especialista en IA, entender la morfología es esencial: un modelo de visión por computador perfectamente entrenado puede ser completamente inútil si el robot físico no tiene los grados de libertad necesarios para actuar sobre lo que percibe.

1.1.1. Configuraciones Mecánicas y Morfología

Dependiendo de la disposición de sus articulaciones (prismáticas para deslizamiento o de revolución para giro), los robots se clasifican en varias tipologías estándar:

- **Robots Manipuladores (Brazos Industriales):** Son cadenas cinemáticas abiertas compuestas por eslabones unidos por articulaciones.
    - *Cartesianos:* Sus ejes coinciden con las coordenadas X, Y, Z. Son fáciles de programar pero ocupan mucho espacio respecto a su área de trabajo.

  - *SCARA (Selective Compliance Assembly Robot Arm):* Muy rápidos en el plano horizontal, ideales para tareas de ensamblaje de electrónica.
  - *Antropomórficos:* Poseen 6 o más grados de libertad (DoF), lo que les permite alcanzar un punto con cualquier orientación. Es la morfología reina en la industria 4.0.

- **Robots Móviles (AMR - Autonomous Mobile Robots):** A diferencia de los brazos, estos no tienen una base fija.
  - *Tracción Diferencial:* Se basan en dos ruedas motrices independientes. Su control es sencillo pero no pueden desplazarse lateralmente.
  - *Omnidireccionales:* Utilizan ruedas especiales (Mecanum) para moverse en cualquier dirección sin cambiar su orientación, lo que optimiza la logística en almacenes inteligentes.

- **Robots Colaborativos (Cobots):** Una evolución crítica para la IA. Están equipados con sensores de par en cada articulación que les permiten detectar colisiones mínimas, permitiendo la coexistencia con humanos sin barreras físicas. Desde el punto de vista del software, los cobots son especialmente interesantes porque su interfaz de programación (*API*) suele estar expuesta a ROS 2 (el sistema operativo de robots que estudiaremos en el Capítulo 2), lo que permite al algoritmo de IA ajustar la "rigidez" del brazo en tiempo real según el contexto: máxima firmeza cuando trabaja solo, mayor suavidad cuando hay personas cerca.

1.1.2. Grados de Libertad (DoF) y Espacio de Trabajo

Un concepto fundamental en el diseño físico es el número de **Grados de Libertad (DoF, del inglés *Degrees of Freedom*)**. Este valor indica cuántos movimientos independientes puede realizar un robot. Una buena analogía es el cuerpo humano: el hombro tiene 3 DoF (puede girar en tres ejes), el codo tiene 1 DoF (solo flexión/extensión) y la muñeca tiene 3 DoF más.

- Para **posicionar** un objeto en el espacio tridimensional se necesitan **3 DoF** (coordenadas x, y, z).
- Para **orientarlo** (ángulos de Euler: *roll* (balanceo), *pitch* (cabeceo), *yaw* (guiñada)) se necesitan otros **3 DoF**.

En total, para alcanzar cualquier punto del espacio con cualquier orientación se necesitan **6 DoF**. Un robot con menos de 6 DoF tiene limitaciones; un robot con más de 6 DoF se considera **redundante**, lo que significa que puede alcanzar un mismo punto con diferentes configuraciones (útil para esquivar obstáculos en el camino).

1.1.3. Opciones de Diseño según el Entorno

El entorno de aplicación condiciona los materiales y la protección del hardware (**Grado IP**):

Entorno	Requisito Crítico	Opción de Diseño Común
Laboratorio / IT	Precisión nanométrica	Actuadores piezoeléctricos y bases de granito.
Almacén Logístico	Autonomía y carga	Chasis de aluminio, LiDAR 360° y baterías LiFePO4.
Industria Química	Estanqueidad	Protecciones IP67 y materiales resistentes a corrosión.
Espacios Humanos	Seguridad (Soft Robotics)	Actuadores elásticos y recubrimientos de espuma.

Tabla 1.1: Relación entre entorno y diseño físico del sistema robotizado.

1.1.4. Laboratorio 1.1: Análisis de Morfologías y DoF

```
TAREA: Identificacion de Sistema
1. Elige un modelo de robot comercial (ej. Universal Robots UR5 o TurtleBot 3).
2. Determina:
   - Tipo de articulaciones (R: Revoluta, P: Prismatica).
   - Numero de Grados de Libertad (DoF).
   - Alcance maximo y Carga util (Payload).
3. Responde: Podria este robot realizar una tarea de soldadura en el techo de
   un coche? Justifica segun sus DoF.
```

1.2. Modelado y Control Cinemático

La cinemática es la rama de la mecánica que estudia el **movimiento** de los sistemas robotizados **sin considerar las fuerzas** que lo originan (eso sería dinámica). Dicho de forma simple: la cinemática responde a la pregunta "¿dónde está la herramienta del robot si sé cómo están girados sus motores?" y a la inversa.

En robótica, este modelado es esencial para realizar la correspondencia entre dos "mundos":

- El **Espacio de Articulación** (q): el mundo interno del robot, definido por los ángulos de sus motores.
- El **Espacio de la Tarea** (x): el mundo real, definido por la posición y orientación del extremo del robot (la herramienta o pinza) en el espacio.

1.2.1. Cinemática Directa (Forward Kinematics)

La cinemática directa responde a la pregunta: **"Sé cómo están girados todos los motores del robot, ¿dónde está la herramienta?"**

Para un manipulador serie, se utiliza el convenio de **Denavit-Hartenberg (D-H)**, que permite sistematizar el cálculo mediante matrices de transformación homogénea (T). Cada eslabón

del robot añade una transformación y el estado final es el producto de todas ellas:

$$T_{total} = T_1 \cdot T_2 \cdot T_3 \cdot \dots \cdot T_n$$

Nota práctica: En el trabajo real con ROS 2, este cálculo no se realiza a mano. Librerías especializadas como **MoveIt2** encapsulan toda la matemática de D-H internamente. La geometría del robot se describe mediante archivos **URDF** (*Unified Robot Description Format*).

1.2.2. Cinemática Inversa (Inverse Kinematics - IK)

La cinemática inversa responde a la pregunta opuesta: **"Quiero que la herramienta esté en esta posición del espacio, ¿cómo debo girar cada motor para lograrlo?"** Presenta tres desafíos principales: múltiples soluciones (codo arriba o abajo), singularidades (el robot pierde movilidad) y complejidad analítica (no siempre existe solución algebraica cerrada).

1.2.3. Control de Trayectorias y el Jacobiano

El **Jacobiano** (J) es una matriz que establece la relación entre las velocidades de las articulaciones $\dot{q}$ y la velocidad cartesiana del extremo $\dot{x}$:

$$\dot{x} = J(q) \cdot \dot{q}$$

Sus dos utilidades principales son el **control de velocidad cartesiana** (trayectorias suaves) y la **detección de singularidades** (cuando $\det(J) \to 0$). La Jacobiana inversa (J^{-1}) resuelve el problema contrario; para robots redundantes se usa la pseudoinversa de Moore-Penrose (J^{+}).

1.2.4. Laboratorio 1.2: Resolución de IK y cálculo numérico del Jacobiano

Entorno de ejecución: entorno Conda del curso.

```
conda activate robots-expertos-mia
python labs/lab_1/lab_1_2a_ik_2d.py
python labs/lab_1/lab_1_2b_jacobiano_numerico.py
```

```python
# Laboratorio 1.2a - Cinematica Inversa 2D
import numpy as np
import matplotlib.pyplot as plt

def resolver_ik_simple(target_x, target_y, l1, l2):
    d_sq = target_x**2 + target_y**2
    d = np.sqrt(d_sq)
    if d > (l1 + l2):
        return None, f"Objetivo ({target_x}, {target_y}) fuera de alcance"
    if d < abs(l1 - l2):
        return None, f"Objetivo ({target_x}, {target_y}) demasiado cercano"
    cos_theta2 = (d_sq - l1**2 - l2**2) / (2 * l1 * l2)
```

```
    theta2 = np.arccos(np.clip(cos_theta2, -1, 1))
    alpha = np.arctan2(target_y, target_x)
    beta  = np.arctan2(l2 * np.sin(theta2), l1 + l2 * np.cos(theta2))
    theta1 = alpha - beta
    return np.degrees([theta1, theta2]), "Exito"

def visualizar_brazo(target_x, target_y, l1, l2, angulos):
    theta1_rad = np.radians(angulos[0])
    theta2_rad = np.radians(angulos[1])
    p0 = np.array([0, 0])
    p1 = p0 + l1 * np.array([np.cos(theta1_rad), np.sin(theta1_rad)])
    p2 = p1 + l2 * np.array([np.cos(theta1_rad+theta2_rad),
                             np.sin(theta1_rad+theta2_rad)])
    fig, ax = plt.subplots(figsize=(6, 6))
    ax.plot([p0[0],p1[0]], [p0[1],p1[1]], 'b-o', linewidth=3, label='Eslabon 1')
    ax.plot([p1[0],p2[0]], [p1[1],p2[1]], 'r-o', linewidth=3, label='Eslabon 2')
    ax.plot(target_x, target_y, 'g*', markersize=15, label='Objetivo (TCP)')
    ax.set_xlim(-l1-l2-1, l1+l2+1); ax.set_ylim(-l1-l2-1, l1+l2+1)
    ax.set_aspect('equal'); ax.grid(True, alpha=0.4); ax.legend()
    ax.set_title(f'IK: theta1={angulos[0]:.1f} | theta2={angulos[1]:.1f} deg')
    plt.tight_layout(); plt.show()

L1, L2 = 10, 10
TARGET_X, TARGET_Y = 10, 10
angulos, msg = resolver_ik_simple(TARGET_X, TARGET_Y, L1, L2)
if angulos is not None:
    print(f"Resultado: {msg}")
    print(f"  -> theta1: {angulos[0]:.2f} grados")
    print(f"  -> theta2: {angulos[1]:.2f} grados")
    visualizar_brazo(TARGET_X, TARGET_Y, L1, L2, angulos)
else:
    print(f"Sin solucion: {msg}")
```

```
# Laboratorio 1.2b - Jacobiano numerico por diferencias finitas
import numpy as np

def cinematica_directa_2d(theta1_deg, theta2_deg, l1, l2):
    t1 = np.radians(theta1_deg); t2 = np.radians(theta2_deg)
    x = l1*np.cos(t1) + l2*np.cos(t1+t2)
    y = l1*np.sin(t1) + l2*np.sin(t1+t2)
    return np.array([x, y])

def calcular_jacobiano_numerico(theta1_deg, theta2_deg, l1, l2, delta_deg=0.01):
    delta_rad = np.radians(delta_deg)
    pos_base  = cinematica_directa_2d(theta1_deg, theta2_deg, l1, l2)
    pos_pert1 = cinematica_directa_2d(theta1_deg+delta_deg, theta2_deg, l1, l2)
    pos_pert2 = cinematica_directa_2d(theta1_deg, theta2_deg+delta_deg, l1, l2)
    col1 = (pos_pert1 - pos_base) / delta_rad
    col2 = (pos_pert2 - pos_base) / delta_rad
    return np.column_stack([col1, col2])

L1, L2 = 10, 10
```

```
T1, T2 = 45.0, 90.0
pos_tcp = cinematica_directa_2d(T1, T2, L1, L2)
J = calcular_jacobiano_numerico(T1, T2, L1, L2)
print(f"TCP: x={pos_tcp[0]:.4f}, y={pos_tcp[1]:.4f}")
print(f"J:\n  [{J[0,0]:8.4f}  {J[0,1]:8.4f}]  <- vel X")
print(f"  [{J[1,0]:8.4f}  {J[1,1]:8.4f}]  <- vel Y")
det_J = np.linalg.det(J)
print(f"det(J) = {det_J:.4f}")
if abs(det_J) < 0.5:
    print("AVISO: proximidad a singularidad!")
else:
    print("OK: alejado de singularidades.")
vel_tcp = np.array([0.1, 0.0])
vel_q   = np.linalg.inv(J) @ vel_tcp
print(f"Para mover TCP a {vel_tcp} m/s:")
print(f"  theta1_dot = {np.degrees(vel_q[0]):.4f} deg/s")
print(f"  theta2_dot = {np.degrees(vel_q[1]):.4f} deg/s")
```

1.3. IA en el Edge: NVIDIA Jetson

Hasta ahora hemos hablado de robots que perciben y actúan. Pero ¿dónde se ejecuta la inteligencia? La respuesta más intuitiva sería "en un servidor en la nube", pero en robótica autónoma esa opción introduce un problema crítico: la **latencia**. Un robot no puede esperar 200 ms a que un servidor remoto procese una imagen para decidir si debe frenar ante un obstáculo que acaba de aparecer.

El concepto de **Edge Computing** (computación en el borde) consiste en desplazar la capacidad de cómputo al propio dispositivo, eliminando la dependencia de la red. La familia **NVIDIA Jetson** es la plataforma líder en este sector: hardware diseñado específicamente para ejecutar redes neuronales profundas con un consumo energético mínimo, directamente a bordo del robot.

1.3.1. Arquitectura SOC y Computación Heterogénea

A diferencia de un PC convencional donde la CPU y la GPU son chips separados conectados por un bus, la Jetson es un **SoC (System on Chip)**: todos los componentes están integrados en una misma placa de circuito. Los componentes principales son:

- **CPU Multi-core (ARM):** Encargada del sistema operativo (Ubuntu), la lógica de control y la gestión de procesos de ROS 2.
- **GPU (Arquitectura Maxwell/Pascal/Ampere):** El núcleo del sistema. Contiene cientos de **CUDA Cores** y **Tensor Cores** (en modelos recientes) optimizados para el cálculo matricial masivo que requieren las redes neuronales.
- **Motores de Codificación/Decodificación de Vídeo:** Permiten procesar múltiples flujos de cámaras de alta resolución en tiempo real sin saturar la CPU.

- **Memoria Unificada:** La CPU y la GPU comparten el mismo espacio de memoria RAM, eliminando el cuello de botella que supone transferir datos entre ambas (crítico en visión artificial).

1.3.2. El Stack de Software: JetPack SDK

El SDK **NVIDIA JetPack** es el paquete de software oficial que se instala en la Jetson e incluye:

CUDA: Plataforma de computación paralela que permite a los desarrolladores usar la GPU para propósitos generales.

cuDNN: Librería de primitivas para redes neuronales profundas.

TensorRT: Un optimizador de inferencia de alto rendimiento. Toma un modelo entrenado (ej. en PyTorch o TensorFlow) y lo "comprime" y optimiza para que corra hasta 10 veces más rápido en el hardware Jetson.

1.3.3. Consumo vs. Rendimiento: el Principio SWaP

En robótica, el diseño se rige por el principio **SWaP** (*Size, Weight, and Power*). Una Jetson Nano o una Orin pueden ejecutar modelos de detección de objetos (como YOLO) consumiendo entre **5W y 15W**. Por comparación, un PC con una tarjeta gráfica equivalente consumiría más de **200W**, haciendo completamente inviable su uso en un robot alimentado por baterías.

1.3.4. Laboratorio 1.3: Benchmarking de Inferencia (CPU vs GPU)

Entorno de ejecución: entorno Conda del curso.

```
conda activate robots-expertos-mia
python labs/lab_1/lab_1_3_benchmark_cpu_gpu.py
```

Nota sobre CUDA en el aula

El entorno Conda del curso instala PyTorch en modo `cpuonly`. En la mayoría de los equipos del aula **solo se ejecutará el bloque de CPU**: el bloque de GPU quedará en `[GPU] CUDA no disponible`. Esto es completamente normal y esperado. El valor pedagógico del ejercicio no son los números en sí, sino **entender por qué la GPU puede ser órdenes de magnitud más rápida** y qué arquitectura hace posible esa diferencia (ver Sección 1.3.2). Si dispones de una GPU NVIDIA y quieres ver la comparativa real, instala la versión CUDA de PyTorch siguiendo las instrucciones en `pytorch.org`.

```
# Laboratorio 1.3 - Benchmark CPU vs GPU con PyTorch
import torch, time, numpy as np

ANCHO, ALTO, ITERACIONES, KERNEL_SIZE = 1920, 1080, 50, 21
```

```
print(f"CUDA disponible: {torch.cuda.is_available()}")

imagen_np = np.random.randint(0, 255, (ALTO, ANCHO, 3), dtype=np.uint8)
imagen_tensor = torch.from_numpy(imagen_np).permute(2,0,1).unsqueeze(0).float()
kernel = torch.ones(3, 1, KERNEL_SIZE, KERNEL_SIZE) / (KERNEL_SIZE**2)

imagen_cpu, kernel_cpu = imagen_tensor.to('cpu'), kernel.to('cpu')
tiempos_cpu = []
for _ in range(ITERACIONES):
    t = time.perf_counter()
    _ = torch.nn.functional.conv2d(imagen_cpu, kernel_cpu,
                                   padding=KERNEL_SIZE//2, groups=3)
    tiempos_cpu.append(time.perf_counter() - t)
media_cpu = np.mean(tiempos_cpu)
print(f"[CPU] {media_cpu*1000:.2f} ms | {1/media_cpu:.1f} FPS")

if torch.cuda.is_available():
    imagen_gpu, kernel_gpu = imagen_tensor.to('cuda'), kernel.to('cuda')
    for _ in range(5):
        _ = torch.nn.functional.conv2d(imagen_gpu, kernel_gpu,
                                       padding=KERNEL_SIZE//2, groups=3)
    torch.cuda.synchronize()
    tiempos_gpu = []
    for _ in range(ITERACIONES):
        torch.cuda.synchronize(); t = time.perf_counter()
        _ = torch.nn.functional.conv2d(imagen_gpu, kernel_gpu,
                                       padding=KERNEL_SIZE//2, groups=3)
        torch.cuda.synchronize()
        tiempos_gpu.append(time.perf_counter() - t)
    media_gpu = np.mean(tiempos_gpu)
    print(f"[GPU] {media_gpu*1000:.2f} ms | {1/media_gpu:.1f} FPS")
    print(f"RESULTADO: GPU es {media_cpu/media_gpu:.1f}x mas rapida")
else:
    print("[GPU] CUDA no disponible. Ver pytorch.org para instalacion con CUDA."
        )
```

1.4. Laboratorio 1: Configuración del Entorno

El objetivo de esta práctica es configurar los dos entornos de software que se usarán a lo largo de todo el curso:

- **Entorno Conda** (`robots-expertos-mia`): para los laboratorios de Python puro (Labs 1 parcial, 3 y 4). Se ejecuta directamente en el PC.
- **Entorno Docker** (imagen `ros2-curso:v2`): para los laboratorios que requieren ROS 2 (Lab 1 validación, Lab 2 y Proyecto Final).

1.4.1. Configuración del Entorno Conda

El entorno Conda incluye Python 3.11 con todas las librerías del curso: `numpy`, `matplotlib`, `scipy`, `scikit-fuzzy`, `pybullet`, `anthropic`, `ollama` y `PyTorch` (modo CPU).

```
# Crear el entorno (solo una vez; puede tardar 3-7 minutos)
conda env create -f environment.yml

# Activar el entorno (necesario en cada terminal nuevo)
conda activate robots-expertos-mia

# Verificar instalacion
python -c "import numpy, matplotlib, skfuzzy, torch, pybullet, anthropic; print
    ('Entorno OK')"
```

1.4.2. ¿Qué es Docker y por qué lo usamos?

El problema: ROS 2 requiere una versión concreta de Ubuntu, una versión concreta de Python, y decenas de librerías con versiones específicas y compatibles entre sí. Si cada alumno instala todo esto directamente sobre su sistema operativo, aparecerán inevitablemente conflictos entre máquinas. En un entorno profesional, instalar software directamente sobre el sistema operativo del servidor es una mala práctica por exactamente las mismas razones.

La solución — los contenedores: Un **contenedor** es una caja sellada e independiente que contiene un sistema operativo completo (Ubuntu), todas las librerías necesarias y nuestra aplicación, todo junto. Esta caja se ejecuta sobre nuestro sistema anfitrión de forma aislada.

Característica	**Instalación directa**	**Contenedor Docker**
Aislamiento	Ninguno	Total
Reproducibilidad	Depende del PC	Idéntico en cualquier máquina
Arranque	Inmediato	Segundos
Eliminar el entorno	Complejo y arriesgado	Un solo comando
Compartir el entorno	Difícil	Enviar un archivo de texto

Tabla 1.2: Comparativa entre instalar software directamente y usar contenedores.

Conceptos clave de Docker:

Imagen (*Image*): La "receta" de solo lectura que define el contenido del contenedor. En este curso usamos una única imagen: `ros2-curso:v2`, que incluye ROS 2 Humble, Gazebo y los paquetes de TurtleBot3.

Contenedor (*Container*): Una instancia en ejecución creada a partir de una imagen. Al detenerlo, los cambios internos se pierden salvo que usemos volúmenes.

Volumen (*Volume*): Un directorio del PC anfitrión montado dentro del contenedor. Así editamos el código en el PC y lo ejecutamos dentro del contenedor sin copiar nada.

`docker compose`: Herramienta para gestionar múltiples servicios a la vez. El repositorio incluye un `docker-compose.yml` con dos servicios (`ros2_v1` y `ros2_v2`) que usan la misma imagen `ros2-curso:v2` pero con diferentes configuraciones de red y pantalla.

1.4.3. Instalación de Docker

Opción A — Linux (Ubuntu 22.04):

```
# Instalar Docker Engine
bash labs/lab_1/shell/01_install_docker_ubuntu.sh

# Opcional: usar Docker sin sudo (requiere reabrir terminal)
bash labs/lab_1/shell/02_enable_docker_without_sudo.sh
```

Opción B — Windows 10/11 con WSL 2:

```
# Paso 1: instalar WSL 2 (PowerShell como Administrador)
wsl --install
# Reiniciar el PC cuando lo solicite.

# Paso 2: instalar Docker Desktop desde:
# https://www.docker.com/products/docker-desktop/
# Marcar "Use WSL 2 instead of Hyper-V" durante la instalacion.

# Paso 3: Docker Desktop > Settings > Resources > WSL Integration
# Activar la integracion con Ubuntu.

# Paso 4: verificar desde terminal Ubuntu (WSL)
docker run hello-world
```

A partir de aquí, **todos los comandos son idénticos en Linux y en WSL 2**.

1.4.4. Construir la imagen Docker del curso

El repositorio incluye un único script que construye la imagen `ros2-curso:v2`. Esta imagen contiene ROS 2 Humble, Gazebo, TurtleBot3 y las librerías Python del curso:

```
# Desde la raiz del repositorio (solo la primera vez; puede tardar 5-10 min)
bash labs/lab_2/shell/01_build_image_v2.sh

# Verificar que la imagen se ha creado:
docker images
# Debe aparecer 'ros2-curso' con tag 'v2'
```

1.4.5. Primer arranque y validación

```
# Levantar el servicio en segundo plano
docker compose up -d ros2_v1
```

```
# Abrir un terminal dentro del contenedor
docker compose exec ros2_v1 bash

# Dentro del contenedor, verificar ROS 2 y librerias:
ros2 --version
python3 -c "import pybullet, skfuzzy, numpy; print('Librerias OK')"

# Salir del contenedor
exit
```

Para la validación completa del entorno (ROS 2 + PyBullet + robot KUKA):

```
# Desde la raiz del repositorio (fuera del contenedor):
bash labs/lab_1/shell/04_validate_env.sh

# Salida esperada:
# [1/4] Python: 3.10.x  OK
# [2/4] ROS 2:  ros2 cli version X.X.X  OK
# [3/4] numpy OK  /  matplotlib OK  /  pybullet OK  /  skfuzzy OK
# [4/4] Robot KUKA cargado. Articulaciones detectadas: 7  OK
```

1.4.6. Referencia rápida de comandos Docker

Comando	**Qué hace**
`docker images`	Lista las imágenes descargadas
`docker ps`	Lista los contenedores en ejecución
`docker ps -a`	Lista todos los contenedores, incluidos los detenidos
`docker compose up -d ros2_v1`	Levanta el servicio base en segundo plano
`docker compose exec ros2_v1 bash`	Abre un terminal en el contenedor en ejecución
`docker compose up -d ros2_v2`	Levanta el servicio con soporte gráfico (Gazebo)
`docker compose exec ros2_v2 bash`	Terminal en el contenedor con Gazebo
`docker compose down`	Detiene y elimina todos los contenedores
`docker rmi ros2-curso:v2`	Elimina la imagen del disco local

Tabla 1.3: Referencia rápida de comandos Docker para el curso.

1.4.7. Entregable del Laboratorio

- **Captura de `docker images`:** Debe mostrar la imagen `ros2-curso:v2`.
- **Captura de la validación completa:** Salida de `04_validate_env.sh` con todos los tests en `OK`.

- **Análisis del Dockerfile:** Abre `labs/lab_2/docker/Dockerfile.v2` y añade un comentario propio en cada instrucción `RUN` explicando con tus propias palabras qué hace.
- **Pregunta de reflexión:** ¿Qué ventaja concreta tendría usar contenedores si el año que viene este curso se actualiza a ROS 2 Iron en lugar de Humble? Responde en 3–4 líneas.

2. Programación y Simulación con ROS 2

En el capítulo anterior entendimos el *cuerpo* del robot: su mecánica, sus motores y su hardware de computación. En este capítulo nos ocupamos del *sistema nervioso*: el software que conecta todos esos componentes y permite que el robot actúe de forma coordinada e inteligente.

El protagonista de este capítulo es **ROS 2** (*Robot Operating System 2*), el estándar industrial para programar robots modernos. A pesar de su nombre, ROS 2 no es un sistema operativo al uso, sino un **middleware** que facilita la comunicación entre los diferentes componentes de un robot: sensores, modelos de IA, controladores de motores, etc.

2.1. Técnicas de Programación: Del Teach Pendant a ROS 2

2.1.1. Programación por Guiado: El Teach Pendant

Históricamente, la robótica industrial se ha basado en el **Teach Pendant**, un terminal de control manual que permite al operario mover el robot punto a punto para grabar trayectorias. Es un sistema rígido: si el entorno cambia un centímetro, el robot falla porque no posee capacidad de percepción ni adaptación.

2.1.2. Programación Estructurada y Lenguajes Propietarios

Con la necesidad de añadir lógica, los fabricantes desarrollaron lenguajes de alto nivel específicos: **RAPID** (ABB), **KRL** (KUKA) o **AS** (Kawasaki). Aunque permitían integrar sensores simples, seguían siendo entornos "monolíticos": integrar librerías de IA como TensorFlow u OpenCV resultaba extremadamente difícil.

2.1.3. El Cambio de Paradigma: Programación basada en Middleware

La robótica moderna requiere una **arquitectura distribuida**: múltiples procesos especializados en paralelo. ROS 2 es el middleware estándar y ofrece tres ventajas fundamentales:

- **Abstracción del Hardware:** El mismo nodo de detección YOLO funciona con un brazo Fanuc o con un TurtleBot móvil.
- **Modularidad (*Nodos*):** Cada función (visión, control, sensorización) es un proceso independiente que se comunica mediante el paso de mensajes.
- **Multilenguaje:** Combina C++ para control de bajo nivel con Python para los modelos de IA.

Característica	Guiado Manual	Estructurada (KRL/RAPID)	Middleware (ROS 2)
Flexibilidad	Muy baja	Media	Muy alta
Integración IA	Imposible	Muy limitada	Nativa y sencilla
Entorno	Estático	Estático con sensores	Dinámico
Paradigma	Secuencial	Procedural	Eventos / Nodos

Tabla 2.1: Comparativa entre métodos de programación robótica.

2.2. Ecosistema ROS 2

2.2.1. Arquitectura Basada en Nodos y el Grafo de Computación

El concepto fundamental de ROS 2 es el **Nodo** (*Node*): un proceso en ejecución que realiza una tarea muy específica. La aplicación robótica completa se compone de múltiples Nodos comunicándose entre sí, formando el **Grafo de Computación**.

2.2.2. Mecanismos de Comunicación: Tópicos, Servicios y Acciones

- **Tópicos:** Comunicación unidireccional y continua basada en Publicador/Suscriptor. Para datos de flujo constante (imágenes de cámara, telemetría de batería).
- **Servicios:** Comunicación bidireccional síncrona basada en Petición/Respuesta. Para tareas rápidas y esporádicas.
- **Acciones:** Comunicación bidireccional asíncrona para tareas de larga duración con feedback de progreso y posibilidad de cancelación.

2.2.3. El estándar de comunicación: DDS

ROS 2 usa **DDS (Data Distribution Service)** como capa de transporte. A diferencia de ROS 1, no existe un proceso centralizado que si falla detiene todo. Los Nodos se descubren entre sí de forma descentralizada (como en una red P2P). DDS permite configurar **QoS (Quality of Service)** por tópico: fiabilidad, durabilidad e historial de cola.

2.2.4. Laboratorio 2.2: Inspección del Grafo y Mensajería

Entorno de ejecución: contenedor Docker.

Este laboratorio requiere **dos terminales simultáneas**, ambas conectadas al mismo contenedor. El esquema de trabajo es el siguiente:

```
+-----------------------------------+  +-----------------------------------+
|  TERMINAL 1                       |  |  TERMINAL 2                       |
|  (publica mensajes)               |  |  (inspecciona el sistema)         |
```

```
+----------------------------------+  +----------------------------------+
| docker compose up -d ros2_v1     |  | docker compose exec ros2_v1 bash |
| docker compose exec ros2_v1 bash |  |                                  |
|                                  |  | ros2 node list                   |
| ros2 run demo_nodes_py talker    |  | ros2 topic list                  |
|   (se queda ejecutando)          |  | ros2 topic echo /chatter         |
|                                  |  | ros2 topic hz /chatter           |
+----------------------------------+  +----------------------------------+
```

El comando `docker compose exec` abre un **terminal adicional en un contenedor ya en ejecución**, sin reiniciarlo. Puedes abrir tantos terminales como necesites sobre el mismo contenedor con este comando.

```
# Referencia de los comandos de inspeccion (Terminal 2):
ros2 node list                          # Debe mostrar: /talker
ros2 topic list                         # Debe mostrar: /chatter
ros2 topic echo /chatter                # Ver datos en tiempo real
ros2 topic hz /chatter                  # Ver frecuencia de publicacion
ros2 interface show std_msgs/msg/String # Ver estructura del mensaje
```

2.3. Anatomía de un Nodo rclpy: Construir desde Cero

Antes de leer o modificar el nodo del Laboratorio 2.4, es imprescindible entender **por qué un nodo tiene la forma que tiene**. Un nodo rclpy no es un script Python secuencial; es una clase que delega el control al ejecutor de ROS 2. Esta diferencia lo cambia todo.

2.3.1. La Plantilla Mínima de un Nodo

Todo nodo de ROS 2 en Python responde a la misma estructura de cinco partes. El siguiente código es el mínimo funcional: un nodo que arranca, se anuncia en el grafo y espera eventos.

```python
# Parte 1 - Importaciones obligatorias
import rclpy
from rclpy.node import Node   # Clase base de la que hereda todo nodo

# Parte 2 - Definicion de la clase
class MiNodo(Node):

    # Parte 3 - Constructor: declarar TODA la infraestructura del nodo
    def __init__(self):
        super().__init__('mi_nodo')   # nombre visible en ros2 node list
        # Aqui se crean publishers, subscribers y timers.
        # NUNCA se escribe logica de negocio aqui: solo infraestructura.
        self.get_logger().info('Nodo iniciado.')

    # Parte 4 - Callbacks: ROS 2 los invoca automaticamente.
    # NUNCA se llaman directamente desde el codigo.

# Parte 5 - Punto de entrada
```

```
def main(args=None):
    rclpy.init(args=args)       # Inicializar el sistema ROS 2
    nodo = MiNodo()             # Instanciar el nodo
    rclpy.spin(nodo)            # Ceder el control al executor (bucle de eventos
        )
    nodo.destroy_node()
    rclpy.shutdown()

if __name__ == '__main__':
    main()
```

2.3.2. Los Tres Elementos de Infraestructura

Dentro del constructor `__init__` se declaran siempre los mismos tres tipos de objetos. Ninguno de ellos ejecuta lógica: solo *registran* qué quiere hacer el nodo.

Publisher Publica mensajes en un tópico.

```
from geometry_msgs.msg import Twist

class MiNodo(Node):
    def __init__(self):
        super().__init__('mi_nodo')
        # create_publisher(TipoMensaje, 'nombre_topico', profundidad_cola)
        self.pub_vel = self.create_publisher(Twist, 'cmd_vel', 10)
```

Subscriber Se suscribe a un tópico y registra qué función (callback) debe invocarse cuando llega un mensaje. El nodo **nunca llama al callback directamente**: es ROS 2 quien lo hace.

```
from sensor_msgs.msg import LaserScan

class MiNodo(Node):
    def __init__(self):
        super().__init__('mi_nodo')
        self.sub_laser = self.create_subscription(
            LaserScan, 'scan', self.callback_laser, 10
        )

    # Este metodo NUNCA se llama desde el codigo: ROS 2 lo invoca solo
    def callback_laser(self, msg):
        distancia_frontal = msg.ranges[0]
        self.get_logger().info(f'Frente: {distancia_frontal:.2f} m')
```

Timer Ejecuta una función periódicamente, sin depender de mensajes externos.

```
class MiNodo(Node):
    def __init__(self):
        super().__init__('mi_nodo')
```

```
        self.timer = self.create_timer(0.5, self.callback_timer)

    def callback_timer(self):
        self.get_logger().info('Tick cada 500 ms')
```

2.3.3. El Rol de `rclpy.spin()`

La llamada `rclpy.spin(nodo)` no es un bucle vacío: es el *executor*, el corazón del sistema de eventos. Mientras `spin()` está activo: monitoriza todos los suscriptores, invoca el callback cuando llega un mensaje, comprueba los timers y devuelve el control solo cuando el proceso recibe `Ctrl+C`.

Patrón mental clave

Un nodo rclpy no hace nada por sí solo. Solo **declara qué le interesa** (suscripciones y timers) y **qué hacer cuando ocurre** (callbacks). Es `rclpy.spin()` quien orquesta todo. El programador escribe *reacciones*, no secuencias.

2.3.4. Tipos de Mensajes Habituales

Tipo	Paquete	Uso habitual
`Twist`	`geometry_msgs.msg`	Comandos de velocidad (`cmd_vel`)
`LaserScan`	`sensor_msgs.msg`	Datos del LiDAR (`/scan`)
`String`	`std_msgs.msg`	Mensajes de texto simples
`Float32`	`std_msgs.msg`	Un único valor numérico
`Odometry`	`nav_msgs.msg`	Posición y velocidad estimadas

Tabla 2.2: Tipos de mensajes más usados en el curso.

```
# Ver la estructura interna de cualquier tipo de mensaje:
ros2 interface show geometry_msgs/msg/Twist
# Vector3  linear  (x, y, z en m/s)
# Vector3  angular (x, y, z en rad/s)
```

2.3.5. El Ecosistema de Nodos al Arrancar el Simulador

Cuando se lanza Gazebo con TurtleBot3, el simulador levanta automáticamente una serie de nodos y tópicos *antes* de que el alumno ejecute su propio código. Entender esta infraestructura es esencial para saber a qué tópicos suscribirse y cuáles publicar.

Nodo activo por defecto	Publica en	Se suscribe a
`/turtlebot3_diff_drive`	`/odom`	`/cmd_vel`
`/turtlebot3_laserscan`	`/scan`	—
`/joint_state_broadcaster`	`/joint_states`	—
`/robot_state_publisher`	`/tf`, `/tf_static`	`/joint_states`
`/gazebo`	`/clock`	—

Tabla 2.3: Nodos y tópicos activos al lanzar la simulación TurtleBot3 en Gazebo.

Lo que esto significa en la práctica: el nodo del alumno **solo necesita** suscribirse a `/scan` (ya publicado por `turtlebot3_laserscan`) y publicar en `/cmd_vel` (ya escuchado por `turtlebot3_diff_drive`). El alumno no implementa ni el driver del sensor ni el driver del motor: solo la lógica de decisión entre ambos.

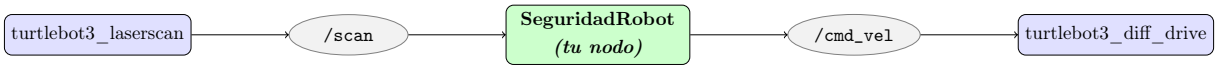

Figura 2.1: El nodo del alumno (verde) se engancha a la infraestructura del simulador (azul).

2.4. Simulación con Gazebo y Webots

Los **simuladores robóticos** permiten probar algoritmos en un entorno virtual seguro, donde un error simplemente se corrige y se vuelve a ejecutar. Para los proyectos de Big Data tienen además otra ventaja: actúan como **generadores de datos sintéticos masivos**.

2.4.1. Gazebo: El Estándar en la Comunidad ROS

Gazebo es el simulador más utilizado con ROS 2 por su integración nativa y su potente motor de física (ODE). Sus plugins publican en Tópicos de ROS 2 exactamente como los sensores reales.

2.4.2. La Transferencia Sim-to-Real

El mayor desafío es el **Reality Gap**: por muy bueno que sea el simulador, el mundo real tiene detalles que no estaban modelados. Para minimizar esta brecha se usan tres técnicas: modelado URDF/SDF de alta fidelidad, **Domain Randomization** (variar aleatoriamente fricción, masa, iluminación durante el entrenamiento) y controladores `ros2_control` que funcionan igual en simulación y en el robot real.

2.4.3. Laboratorio 2.3: Lanzamiento y Navegación en Mundo Virtual

Entorno de ejecución: contenedor Docker (servicio `ros2_v2`, con soporte gráfico).

Nota sobre Gazebo y entornos sin pantalla: En el aula con Linux funciona directamente. En WSL 2 es necesario tener WSLg activo (Windows 11) o VcXsrv (Windows 10). Los tópicos

ROS 2 (incluyendo `/scan`) funcionan aunque la ventana gráfica no se muestre.

```
# Terminal 1: levantar el servicio con soporte grafico y lanzar Gazebo
bash labs/lab_2/shell/02_run_turtlebot3_sim.sh
# Dentro del contenedor ros2_v2:
ros2 launch turtlebot3_gazebo turtlebot3_world.launch.py

# Terminal 2: comprobar que /scan esta publicando
bash labs/lab_2/shell/03_check_scan_topics.sh
```

Si aparece `WARNING: topic [/scan] does not appear to be published yet`, Gazebo todavía no ha terminado de arrancar. Esperar 10–20 segundos y repetir.

2.5. Laboratorio 2: Mi primer nodo de ROS 2 en Python

En esta práctica final del capítulo, el alumno desarrollará un *Nodo* de control utilizando `rclpy`: un comportamiento de "Evasión de Obstáculos" reactivo.

2.5.1. Estructura de un Nodo en Python

Un *Nodo* de ROS 2 en Python se estructura como una clase que hereda de `rclpy.node.Node`. El concepto de **callback** es central: es una función que **no llamamos nosotros directamente**, sino que ROS 2 invoca automáticamente cada vez que ocurre un evento. La función `rclpy.spin(nodo)` mantiene el *Nodo* activo escuchando esos eventos (ver Sección 2.3 para la anatomía completa).

2.5.2. Paso Previo: El Nodo "LectorLaser" (nodo puente)

Antes de enfrentarse al nodo completo de seguridad, que integra varias capas de lógica simultáneas, es recomendable construir y ejecutar este nodo puente de 25 líneas. Su único objetivo es **leer el sensor LiDAR y publicar la distancia mínima frontal**: no toma ninguna decisión, no mueve el robot. Esto permite verificar que el entorno funciona correctamente y que los datos del sensor son coherentes antes de añadir complejidad.

```python
# nodo_lector_laser.py  -  nodo puente de verificacion
# Ejecutar: python3 /workspace_repo/labs/lab_2/nodo_lector_laser.py
#
# Que hace: suscribe /scan y publica la distancia minima frontal
# en /distancia_frontal como Float32. No mueve el robot.
# Util para verificar que el sensor funciona antes de anadir logica.

import math
import rclpy
from rclpy.node import Node
from sensor_msgs.msg import LaserScan
from std_msgs.msg import Float32
```

```python
class LectorLaser(Node):
    def __init__(self):
        super().__init__('lector_laser')

        # Subscriber: escucha el sensor LiDAR
        self.sub = self.create_subscription(
            LaserScan, 'scan', self.callback_scan, 10
        )
        # Publisher: publica la distancia minima frontal
        self.pub = self.create_publisher(Float32, 'distancia_frontal', 10)

        self.get_logger().info('LectorLaser iniciado. Escuchando /scan...')

    def callback_scan(self, msg):
        n = len(msg.ranges)
        # Sector frontal: primeros y ultimos 10 rayos del array
        # (en TurtleBot3, el frente esta en los indices 0 y -1)
        sector_frontal = list(msg.ranges[:10]) + list(msg.ranges[-10:])

        # Filtrar valores invalidos (NaN, inf, fuera de rango del sensor)
        validos = [v for v in sector_frontal
                   if math.isfinite(v) and msg.range_min < v < msg.range_max]

        dist_min = min(validos) if validos else msg.range_max

        msg_out = Float32()
        msg_out.data = dist_min
        self.pub.publish(msg_out)

        self.get_logger().info(f'Distancia frontal minima: {dist_min:.3f} m')


def main(args=None):
    rclpy.init(args=args)
    nodo = LectorLaser()
    rclpy.spin(nodo)
    nodo.destroy_node()
    rclpy.shutdown()

if __name__ == '__main__':
    main()
```

Para verificar que funciona, abre dos terminales adicionales:

```
# Terminal A: ejecutar el nodo puente (con el simulador ya en marcha)
docker compose exec ros2_v1 bash
python3 /workspace_repo/labs/lab_2/nodo_lector_laser.py

# Terminal B: confirmar que el topico existe y tiene datos
docker compose exec ros2_v1 bash
ros2 topic echo /distancia_frontal
# Debe mostrar valores numericos que cambian al mover objetos
```

```
# frente al robot en la simulacion (entre 0.1 y 3.5 m aprox.).
```

Una vez que `/distancia_frontal` muestre valores coherentes, el entorno está listo para el nodo completo. **La única diferencia entre este nodo puente y el `SeguridadRobot` es la lógica del callback**: la estructura del nodo (clase, constructor, subscriber, publisher, spin) es exactamente la misma.

2.5.3. Implementación: El Nodo "SeguridadRobot"

Entorno de ejecución: contenedor Docker (servicio `ros2_v1`). Para que funcione, el Laboratorio 2.3 debe estar en marcha (Gazebo publicando `/scan`).

El nodo implementado en el repositorio incluye: sectores angulares dinámicos, maniobra de recuperación en dos fases (marcha atrás + giro), control de velocidad proporcional y protección lateral contra paredes.

```
# Con el simulador ya levantado (Lab 2.3):
docker compose up -d ros2_v1
docker compose exec ros2_v1 bash
# Dentro del contenedor:
python3 /workspace_repo/labs/lab_2/lab_2_4_nodo_seguridad.py
```

```python
# Laboratorio 2.4 - Nodo de Seguridad con ROS 2
import math
import rclpy
from geometry_msgs.msg import Twist
from rclpy.node import Node
from sensor_msgs.msg import LaserScan


class SeguridadRobot(Node):
    def __init__(self):
        super().__init__("nodo_seguridad")
        self.publisher_ = self.create_publisher(Twist, "cmd_vel", 10)
        self.subscription = self.create_subscription(
            LaserScan, "scan", self.listener_callback, 10
        )
        self.stop_dist  = 0.40;  self.slow_dist  = 0.85;  self.clear_dist = 1.20
        self.max_linear = 0.12;  self.min_linear = 0.04;  self.max_angular = 1.2
        self.side_safe_dist = 0.26;  self.front_side_safe_dist = 0.30
        self.turning_side = 1.0
        self.recovery_until_ns = 0;  self.recovery_phase = "none"
        self.reverse_duration_ns = int(0.45 * 1e9)
        self.turn_duration_ns    = int(0.85 * 1e9)
        self.get_logger().info("Nodo SeguridadRobot iniciado.")

    @staticmethod
    def _valid_ranges(values, rmin, rmax):
        return [v for v in values if math.isfinite(v) and rmin < v < rmax]
    @staticmethod
    def _safe_min(values, default): return min(values) if values else default
```

```
    @staticmethod
    def _safe_avg(values, default):
        return sum(values)/len(values) if values else default

    def listener_callback(self, msg):
        now_ns = self.get_clock().now().nanoseconds
        n = len(msg.ranges)
        if n < 10: return

        front_span = max(10, int(0.08*n)); side_span = max(20, int(0.18*n))
        fside_span = max(14, int(0.12*n))
        raw_front = list(msg.ranges[:front_span]) + list(msg.ranges[-front_span
            :])
        raw_fl = list(msg.ranges[front_span:front_span+fside_span])
        raw_fr = list(msg.ranges[n-front_span-fside_span:n-front_span])
        raw_l  = list(msg.ranges[int(0.25*n):int(0.25*n)+side_span])
        raw_r  = list(msg.ranges[int(0.75*n):int(0.75*n)+side_span])

        vr = lambda x: self._valid_ranges(x, msg.range_min, msg.range_max)
        d_front = self._safe_min(vr(raw_front), msg.range_max)
        d_fl    = self._safe_min(vr(raw_fl),    msg.range_max)
        d_fr    = self._safe_min(vr(raw_fr),    msg.range_max)
        d_left  = self._safe_avg(vr(raw_l),     msg.range_max)
        d_right = self._safe_avg(vr(raw_r),     msg.range_max)

        cmd = Twist()
        if self.recovery_phase == "reverse" and now_ns >= self.recovery_until_ns
            :
            self.recovery_phase = "turn"
            self.recovery_until_ns = now_ns + self.turn_duration_ns
        elif self.recovery_phase == "turn" and now_ns >= self.recovery_until_ns:
            self.recovery_phase = "none"

        if self.recovery_phase != "none" and now_ns < self.recovery_until_ns:
            if self.recovery_phase == "reverse":
                cmd.linear.x, cmd.angular.z = -0.06, 0.0
            else:
                cmd.linear.x, cmd.angular.z = 0.0, self.turning_side*self.
                    max_angular
            self.publisher_.publish(cmd); return

        if (d_front < self.stop_dist or d_fl < self.front_side_safe_dist
                or d_fr < self.front_side_safe_dist):
            self.turning_side = 1.0 if d_left >= d_right else -1.0
            self.recovery_phase = "reverse"
            self.recovery_until_ns = now_ns + self.reverse_duration_ns
            cmd.linear.x, cmd.angular.z = -0.06, 0.0
            self.get_logger().warn(f"Obstaculo critico ({d_front:.2f}m). Marcha
                atras.")
        elif d_front < self.slow_dist:
            ratio = max(0.0, min(1.0,
                (d_front-self.stop_dist)/(self.slow_dist-self.stop_dist)))
```

```
            cmd.linear.x  = self.min_linear + ratio*(self.max_linear-self.
                min_linear)
            cmd.angular.z = max(-self.max_angular,
                                min(self.max_angular, 2.0*(d_left-d_right)))
        else:
            cmd.linear.x  = self.max_linear
            cmd.angular.z = max(-0.35, min(0.35, 0.6*(d_left-d_right)))
            if d_front > self.clear_dist: cmd.angular.z *= 0.5

        if d_left < self.side_safe_dist and d_right >= self.side_safe_dist:
            cmd.linear.x = min(cmd.linear.x, 0.04)
            cmd.angular.z = min(cmd.angular.z, -0.70)
        elif d_right < self.side_safe_dist and d_left >= self.side_safe_dist:
            cmd.linear.x = min(cmd.linear.x, 0.04)
            cmd.angular.z = max(cmd.angular.z, 0.70)
        elif min(d_left, d_right) < self.side_safe_dist:
            cmd.linear.x = min(cmd.linear.x, 0.03); cmd.angular.z *= 0.6

        self.publisher_.publish(cmd)


def main(args=None):
    rclpy.init(args=args)
    nodo = SeguridadRobot()
    rclpy.spin(nodo)
    nodo.destroy_node()
    rclpy.shutdown()

if __name__ == "__main__":
    main()
```

2.5.4. El flujo de ejecución en el laboratorio

1. **Lanzar el simulador:** Servicio `ros2_v2` con Gazebo + TurtleBot3 (Lab 2.3).
2. **Verificar el sensor:** Ejecutar el nodo puente `LectorLaser` y comprobar que `/distancia_frontal` recibe datos coherentes.
3. **Ejecutar el Nodo principal:** Servicio `ros2_v1`, correr `lab_2_4_nodo_seguridad.py`. Se conectará automáticamente al grafo.
4. **Monitorización:** Abrir otro terminal con `docker compose exec ros2_v1 bash` y ejecutar `ros2 topic echo /cmd_vel`.
5. **Visualización:** Usar `rqt_graph` para ver el grafo completo y RViz2 para el láser.

2.5.5. Entregable del Laboratorio

- **Optimización de Lógica:** Modificar el código para que el robot gire hacia el lado con más espacio libre, comparando la media de los 30 grados izquierdos vs los 30 derechos en

`msg.ranges`.

- **Análisis del Grafo:** Captura de pantalla de `rqt_graph` con la conexión entre simulador y Nodo.
- **Estudio de Latencia:** Explicar qué ocurriría si la frecuencia de publicación fuera 1 Hz en un entorno con obstáculos móviles.

3. Sistemas Expertos y Lógica de Decisión

Los capítulos anteriores nos han dado las herramientas para que el robot perciba su entorno (sensores, LiDAR, cámaras) y se comunique internamente (ROS 2). Ahora llega la pregunta más importante: **¿cómo decide el robot qué hacer?**

Este capítulo presenta la primera gran familia de técnicas de IA aplicada a robótica: la **IA Simbólica**, y en particular los **Sistemas Expertos**. A diferencia del Deep Learning —que aprende de miles de ejemplos y opera como una caja negra— los Sistemas Expertos funcionan con reglas lógicas escritas por humanos, cuyo razonamiento es completamente explicable y auditable.

3.1. Introducción a los Sistemas Expertos (SE)

Un Sistema Experto (SE) es un programa de IA diseñado para resolver problemas complejos en un dominio específico, imitando la toma de decisiones de un experto humano. La diferencia clave con el Machine Learning tradicional es que **el conocimiento es explícito**: está escrito en forma de reglas que cualquier persona puede leer, comprender y auditar.

3.1.1. Definición y Filosofía

La filosofía de un SE se basa en la **separación entre el conocimiento y el motor que lo ejecuta**. El conocimiento reside en una **Base de Conocimiento** independiente; un motor separado la interpreta. Un experto del dominio puede añadir o modificar reglas *sin tocar una sola línea del motor de inferencia.*

Desde la perspectiva del Big Data, los SE tienen una ventaja clave: **funcionan desde el primer día, sin datos históricos**. Un modelo de ML necesita meses de datos etiquetados; un SE puede estar operativo desde el momento en que el experto codifica su conocimiento.

3.1.2. Tipos de Sistemas Expertos

Basados en Reglas (Rule-Based): El conocimiento se almacena como reglas *SI <condición> ENTONCES <acción>*. Los más comunes en control robótico por su transparencia.

Basados en Casos (CBR): Resuelven problemas nuevos buscando el caso más similar en una base de datos histórica.

Basados en Redes Bayesianas: Introducen la incertidumbre y la probabilidad, permitiendo tomar decisiones con datos ruidosos o incompletos.

3.1.3. Laboratorio 3.1: Identificación de Reglas de Experto

```
EJERCICIO: Extraccion de Conocimiento.
Protocolo: "Si el LED parpadea en rojo, comprueba el voltaje. Si el voltaje
es inferior a 12V, la fuente falla. Si el voltaje es correcto pero el
parpadeo sigue, la temperatura podria ser excesiva (>90C)."

TAREA: Redacta este protocolo como Hechos y Reglas formales (IF-THEN):
Hecho observable: led == 'rojo'
Regla 1: IF (led == 'rojo') AND (voltaje < 12) THEN diagnostico = 'Fallo Fuente'
Regla 2: ...
Regla 3: ...
```

3.2. Estructura de un Sistema Experto

3.2.1. Base de Conocimiento, Motor de Inferencia y Justificador

Un SE tiene arquitectura tripartita: **Base de Conocimiento** (hechos y reglas), **Motor de Inferencia** (examina hechos y dispara reglas) y **Justificador** (explica qué reglas se activaron y por qué). El Justificador es el componente que otorga la **Explicabilidad (XAI)**, crítica en medicina, mantenimiento industrial y robótica con personas.

3.2.2. Laboratorio 3.2: Representación de Hechos y Reglas

Entorno: Conda.

```
conda activate robots-expertos-mia
python labs/lab_3/lab_3_2_motor_inferencia_basico.py
```

```python
# Laboratorio 3.2 - Motor de Inferencia basico
memoria_trabajo = {
    "temperatura_gpu": 85, "carga_cpu": 95,
    "robot_movimiento": True, "operario_cerca": False,
}
base_conocimiento = [
    {"nombre": "Alerta sobrecalentamiento",
     "condicion": lambda f: f["temperatura_gpu"] > 80 and f["robot_movimiento"],
     "accion": "Reducir velocidad de motores y activar ventiladores al maximo"},
    {"nombre": "Modo ahorro de energia",
     "condicion": lambda f: not f["robot_movimiento"] and f["carga_cpu"] < 10,
     "accion": "Entrar en estado de suspension"},
    {"nombre": "Parada por seguridad humana",
     "condicion": lambda f: f["operario_cerca"] and f["robot_movimiento"],
     "accion": "Reducir velocidad al 30% y activar modo colaborativo"},
]
print("--- Motor de Inferencia ---")
alguna = False
for r in base_conocimiento:
    if r["condicion"](memoria_trabajo):
        print(f"[DISPARADA] {r['nombre']} -> {r['accion']}")
```

```
        alguna = True
if not alguna:
    print("Ninguna regla activada. Sistema en estado nominal.")
```

3.3. Estrategias de Control y Simulación

3.3.1. Encadenamiento hacia Adelante (Forward Chaining)

Estrategia **dirigida por los datos**: parte de los Hechos observados y va disparando reglas en cascada. Uso en robótica: control en tiempo real y monitorización continua. Ejemplo: se detecta "humo" → se infiere "fuego" → se activa "Aspersores".

3.3.2. Encadenamiento hacia Atrás (Backward Chaining)

Estrategia **dirigida por los objetivos**: parte de una hipótesis y trabaja hacia atrás buscando hechos que la sustenten. Uso en robótica: diagnóstico de fallos y planificación de tareas. Ejemplo: meta "¿Por qué el robot no se mueve?" → ¿Falla la batería? → ¿Hay obstrucción mecánica?

3.3.3. Resolución de Conflictos

Cuando múltiples reglas pueden dispararse simultáneamente el Motor aplica: **Prioridad explícita** (reglas de seguridad tienen siempre prioridad máxima), **Especificidad** (la regla con más condiciones cumplidas prevalece) y **Recencia** (se priorizan reglas que usan hechos añadidos recientemente).

3.3.4. Laboratorio 3.3: Diagnóstico mediante Backward Chaining

Entorno: Conda.

```
conda activate robots-expertos-mia
python labs/lab_3/lab_3_3_backward_chaining.py
```

```
# Laboratorio 3.3 - Backward Chaining
reglas_diagnostico = {
    "Fallo_Sistema":      ["Fallo_Alimentacion", "Fallo_Software"],
    "Fallo_Alimentacion": ["Bateria_Vacia", "Cable_Desconectado"],
    "Fallo_Software":     ["Error_Kernel", "Proceso_ROS_Caido"],
}
hechos_reales = {"Proceso_ROS_Caido"}
cadena = []

def verificar_meta(meta, prof=0):
    s = "  " * prof
    cadena.append(f"{s}Verificando: '{meta}'")
    if meta in hechos_reales:
```

```
        cadena.append(f"{s}  -> VERDADERO (hecho observado)"); return True
    if meta in reglas_diagnostico:
        for sub in reglas_diagnostico[meta]:
            if verificar_meta(sub, prof+1):
                cadena.append(f"{s}  -> VERDADERO (via '{sub}')"); return True
    cadena.append(f"{s}  -> FALSO"); return False

objetivo = "Fallo_Sistema"
resultado = verificar_meta(objetivo)
for linea in cadena: print(linea)
print(f"\n'{objetivo}' es {'CONFIRMADO' if resultado else 'NO CONFIRMADO'}")
```

3.4. Controladores Inteligentes: Lógica Difusa

La **Lógica Difusa** (*Fuzzy Logic*) permite trabajar con **valores de verdad graduales** en lugar de los absolutos booleanos. En el mundo real, pocas cosas son completamente ciertas: un motor puede estar "bastante caliente" (0.8) o "moderadamente frío" (0.3). Esta capacidad la convierte en una de las técnicas más utilizadas en control robótico, especialmente cuando se necesitan transiciones suaves.

3.4.1. Los Tres Pasos del Control Difuso

1. **Fuzzificación:** Transforma valores numéricos de sensores en grados de pertenencia a conjuntos difusos. Una temperatura de 75°C puede pertenecer a "normal" con grado 0.4 *y* a "alta" con grado 0.6 simultáneamente.

2. **Inferencia Difusa:** Evalúa las reglas lingüísticas sobre los grados de pertenencia. El AND difuso usa el mínimo, el OR usa el máximo.

3. **Defuzzificación:** Convierte el conjunto difuso resultante en un único valor numérico concreto para el actuador. El método más común es el **centroide**: centro de masa del área del conjunto difuso de salida.

3.4.2. Laboratorio 3.4: Controlador Difuso de Velocidad

Entorno: Conda.

```
conda activate robots-expertos-mia
python labs/lab_3/lab_3_4_controlador_difuso.py
```

```
# Laboratorio 3.4 - Controlador Difuso de Velocidad
import numpy as np, skfuzzy as fuzz
from skfuzzy import control as ctrl
import matplotlib.pyplot as plt

distancia = ctrl.Antecedent(np.arange(0, 3.1, 0.01), "distancia")
velocidad = ctrl.Consequent(np.arange(0, 0.51, 0.01), "velocidad")
```

```
distancia["muy_cerca"] = fuzz.trimf(distancia.universe, [0.0, 0.0, 0.5])
distancia["cerca"]     = fuzz.trimf(distancia.universe, [0.2, 0.6, 1.0])
distancia["media"]     = fuzz.trimf(distancia.universe, [0.7, 1.2, 1.7])
distancia["lejos"]     = fuzz.trimf(distancia.universe, [1.4, 2.0, 2.6])
distancia["muy_lejos"] = fuzz.trimf(distancia.universe, [2.3, 3.0, 3.0])

velocidad["parado"] = fuzz.trimf(velocidad.universe, [0.00, 0.00, 0.05])
velocidad["lento"]  = fuzz.trimf(velocidad.universe, [0.03, 0.10, 0.20])
velocidad["medio"]  = fuzz.trimf(velocidad.universe, [0.15, 0.25, 0.35])
velocidad["rapido"] = fuzz.trimf(velocidad.universe, [0.30, 0.40, 0.50])
velocidad["max"]    = fuzz.trimf(velocidad.universe, [0.45, 0.50, 0.50])

sistema = ctrl.ControlSystem([
    ctrl.Rule(distancia["muy_cerca"], velocidad["parado"]),
    ctrl.Rule(distancia["cerca"],     velocidad["lento"]),
    ctrl.Rule(distancia["media"],     velocidad["medio"]),
    ctrl.Rule(distancia["lejos"],     velocidad["rapido"]),
    ctrl.Rule(distancia["muy_lejos"], velocidad["max"]),
])
sim = ctrl.ControlSystemSimulation(sistema)

print(f"{'Distancia (m)':<18}{'Velocidad (m/s)':<18}{'Interpretacion'}")
for d in [0.1, 0.4, 0.8, 1.5, 2.5]:
    sim.input["distancia"] = d; sim.compute(); v = sim.output["velocidad"]
    interp = ("PARADO" if v < 0.05 else
              "lento"  if v < 0.20 else
              "medio"  if v < 0.35 else "alto")
    print(f"{d:<18.1f}{v:<18.3f}{interp}")

rango = np.arange(0.05, 3.0, 0.05); vels = []
for d in rango:
    sim.input["distancia"] = d; sim.compute()
    vels.append(sim.output["velocidad"])
fig, ax = plt.subplots(figsize=(9, 4))
ax.plot(rango, vels, "b-", linewidth=2)
ax.fill_between(rango, 0, vels, alpha=0.15)
ax.axvline(x=0.5, color="r", linestyle="--", alpha=0.6,
           label="Umbral critico (0.5m)")
ax.set_xlabel("Distancia al obstaculo (m)")
ax.set_ylabel("Velocidad (m/s)")
ax.set_title("Curva de respuesta del controlador difuso")
ax.grid(True, alpha=0.4); ax.legend()
plt.tight_layout(); plt.show()
```

3.5. Laboratorio 3: SE para Diagnóstico de Robots

3.5.1. Implementación: Motor de Diagnóstico

Entorno: Conda.

```
conda activate robots-expertos-mia
python labs/lab_3/lab_3_5_sistema_experto_salud.py
```

```python
# Laboratorio 3.5 - Sistema Experto de Salud Robotica
class SistemaExpertoSalud:
    def __init__(self):
        self.memoria_hechos = {}; self.telemetria = {}; self.diagnosticos = []

    def actualizar_hechos(self, t):
        self.telemetria = t
        self.memoria_hechos["v_bajo"]          = t["voltaje"]     < 11.0
        self.memoria_hechos["temp_alta"]        = t["temperatura"] > 85.0
        self.memoria_hechos["perdida_paquetes"] = t["latencia"]    > 100
        self.memoria_hechos["corriente_alta"]   = t["amperaje"]    > 5.0

    def inferir(self):
        self.diagnosticos = []; t = self.telemetria
        no_c = lambda excl: [k for k, v in self.memoria_hechos.items()
                             if not v and k not in excl]

        if self.memoria_hechos["v_bajo"] and self.memoria_hechos["corriente_alta
            "]:
            self.diagnosticos.append({
                "nivel": "FALLO CRITICO", "mensaje": "Posible cortocircuito.",
                "evidencia": [f"Voltaje ({t['voltaje']}V < 11V)",
                              f"Amperaje ({t['amperaje']}A > 5A)"],
                "no_cumplidas": no_c(["v_bajo", "corriente_alta"])})
        if self.memoria_hechos["temp_alta"] and self.memoria_hechos["
            corriente_alta"]:
            self.diagnosticos.append({
                "nivel": "MANTENIMIENTO", "mensaje": "Esfuerzo mecanico excesivo
                    .",
                "evidencia": [f"Temp ({t['temperatura']}C > 85C)",
                              f"Amperaje ({t['amperaje']}A > 5A)"],
                "no_cumplidas": no_c(["temp_alta", "corriente_alta"])})
        if self.memoria_hechos["perdida_paquetes"] and self.memoria_hechos["
            temp_alta"]:
            self.diagnosticos.append({
                "nivel": "FALLO LOGICO", "mensaje": "Throttling en Jetson.",
                "evidencia": [f"Latencia ({t['latencia']}ms > 100ms)",
                              f"Temp ({t['temperatura']}C > 85C)"],
                "no_cumplidas": no_c(["perdida_paquetes", "temp_alta"])})
        return self.diagnosticos

se = SistemaExpertoSalud()
log = {"voltaje": 10.5, "temperatura": 92.0, "latencia": 120, "amperaje": 6.2}
se.actualizar_hechos(log); alertas = se.inferir()
print("=" * 50); print("REPORTE DEL SISTEMA EXPERTO"); print("=" * 50)
if not alertas:
    print("[OK] Sistema nominal.")
else:
    for d in alertas:
```

```
        print(f"[{d['nivel']}] {d['mensaje']}")
        print(f"  Evidencia: {' Y '.join(d['evidencia'])}")
        if d["no_cumplidas"]:
            print(f"  No cumplidas: {d['no_cumplidas']}")
```

3.5.2. Entregable del Laboratorio

1. **Ampliación de la Base de Conocimiento:** Añadir al menos 3 reglas adicionales que involucren sensores LiDAR y estado del software (`proceso_ia_caido`).
2. **Simulación de Avería:** Script con `random` que genere telemetría aleatoria y verifique que el SE identifica correctamente las combinaciones de fallos.
3. **Integración Difusa:** Modificar `actualizar_hechos()` para que el umbral de `temp_alta` use el controlador difuso del Laboratorio 3.4 y dispare la regla solo si el grado de pertenencia supera 0.6.

4. Integración: El Robot Autónomo Inteligente

Los tres capítulos anteriores han construido las piezas del puzzle: el hardware y la mecánica (Cap. 1), el sistema de comunicación entre procesos (Cap. 2) y el motor de razonamiento lógico (Cap. 3). Este capítulo ensambla todas esas piezas en un sistema cohesionado y aborda los problemas que solo aparecen cuando el robot opera en el **mundo real**.

4.1. De Python puro a ROS 2: Integrar el SE en un Nodo

Los Laboratorios 3.2 a 3.5 han construido el Sistema Experto como una **clase Python pura**, ejecutada directamente desde la terminal con `conda activate`. El Proyecto Final exige integrarlo dentro de un **nodo ROS 2** que reciba datos de sensores por tópicos y publique decisiones hacia otros nodos.

Ese paso —de clase Python a nodo ROS 2— no es complicado, pero requiere entender el patrón exacto. El principio es sencillo: **el SE se instancia en el constructor del nodo y se llama desde el callback**. El nodo actúa como envoltorio de comunicación; el SE conserva toda su lógica intacta.

```
# Patron: Sistema Experto dentro de un nodo ROS 2
# Este es el esqueleto que todos los grupos del proyecto final
# deben seguir para integrar su SE con ROS 2.

import json
import rclpy
from rclpy.node import Node
from std_msgs.msg import String

# Importar el SE del laboratorio 3.5 (misma clase, sin modificar)
from lab_3_5_sistema_experto_salud import SistemaExpertoSalud


class NodoCerebroSE(Node):
    """
    - Suscribe: /telemetria  (String JSON con datos de sensores)
    - Publica:  /modo_operacion (String con la decision del SE)
    - Publica:  /justificacion  (String con la explicacion)
    """

    def __init__(self):
        super().__init__('nodo_cerebro_se')

        # PASO 1: Instanciar el SE en el constructor.
        self.se = SistemaExpertoSalud()

        # PASO 2: Subscriber de entrada
```

```
        self.sub_telemetria = self.create_subscription(
            String, 'telemetria', self.callback_telemetria, 10
        )

        # PASO 3: Publishers de salida
        self.pub_modo          = self.create_publisher(String, 'modo_operacion',
             10)
        self.pub_justificacion = self.create_publisher(String, 'justificacion',
            10)

        self.get_logger().info('NodoCerebroSE listo. Esperando telemetria...')

    def callback_telemetria(self, msg):
        # PASO 4: Parsear el JSON recibido
        try:
            datos = json.loads(msg.data)
        except json.JSONDecodeError:
            self.get_logger().warn('JSON de telemetria invalido, ignorando.')
            return

        # PASO 5: Actualizar el SE y ejecutar inferencia
        self.se.actualizar_hechos(datos)
        alertas = self.se.inferir()

        # PASO 6: Determinar el modo de operacion
        if not alertas:
            modo = 'NOMINAL'
            justificacion = 'Sistema en estado nominal. Sin alertas activas.'
        else:
            orden = ['FALLO CRITICO', 'FALLO LOGICO', 'MANTENIMIENTO']
            niveles_activos = [a['nivel'] for a in alertas]
            modo = next((n for n in orden if n in niveles_activos), '
                MANTENIMIENTO')
            alerta_principal = next(a for a in alertas if a['nivel'] == modo)
            justificacion = (
                f"[{modo}] {alerta_principal['mensaje']} "
                f"Evidencia: {' Y '.join(alerta_principal['evidencia'])}"
            )

        # PASO 7: Publicar la decision y la justificacion
        self.pub_modo.publish(String(data=modo))
        self.pub_justificacion.publish(String(data=justificacion))
        self.get_logger().info(f'Modo: {modo} | {justificacion}')


def main(args=None):
    rclpy.init(args=args)
    nodo = NodoCerebroSE()
    rclpy.spin(nodo)
    nodo.destroy_node()
    rclpy.shutdown()
```

```
if __name__ == '__main__':
    main()
```

Los siete pasos marcados en el código son el **patrón completo**. Todos los grupos del proyecto final seguirán exactamente esta misma estructura, variando únicamente: la clase SE importada, los nombres de los tópicos y la lógica de decisión del callback. La estructura del nodo en sí no cambia.

Regla de oro para el proyecto final

El SE no sabe que existe ROS 2. El nodo no sabe cómo funciona el SE. El nodo solo convierte mensajes ROS en datos Python (en el callback) y datos Python en mensajes ROS (al publicar). El SE solo recibe un diccionario y devuelve una lista. Esta separación de responsabilidades hace el sistema mantenible y testeable de forma independiente.

4.2. Problemas Comunes en Robótica y Soluciones de IA

4.2.1. Navegación en Entornos Dinámicos: SLAM

SLAM (*Simultaneous Localization and Mapping*): ¿cómo puede el robot construir un mapa del entorno y saber dónde está dentro de ese mapa al mismo tiempo? Los entornos reales cambian constantemente. La solución son algoritmos de **Filtro de Partículas** o **Grafos de Optimización** que discriminan entre obstáculos estáticos y dinámicos.

Conexión con Big Data: El resultado del SLAM es un **mapa de ocupación** (*occupancy grid map*): una matriz donde cada celda representa la probabilidad de estar ocupada. Un mapa de almacén mediano genera matrices de miles por miles de celdas actualizadas a decenas de Hz: un flujo de datos en tiempo real de alta dimensionalidad.

4.2.2. Gestión de la Incertidumbre Sensorial

Los sensores físicos nunca son perfectos. **Fusión Sensorial:** combinar datos de múltiples fuentes (IMU, Odometría, LiDAR, Cámaras) para obtener una estimación más fiable. El **Filtro de Kalman Extendido (EKF)** es el algoritmo estándar en robótica para fusionar sensores.

4.2.3. Laboratorio 4.1: Análisis de Incertidumbre en la Odometría

Entorno: Conda.

```
conda activate robots-expertos-mia
python labs/lab_4/lab_4_1_deriva_odometria.py
```

```
# Laboratorio 4.1 - Deriva en odometria
import matplotlib.pyplot as plt, numpy as np
np.random.seed(42)
```

```
N_PASOS, RUIDO_SIGMA = 100, 0.05

angulos = np.linspace(0, 2*np.pi, N_PASOS)
real_x, real_y = np.cos(angulos)*5, np.sin(angulos)*5
est_x, est_y = [real_x[0]], [real_y[0]]
for i in range(1, N_PASOS):
    est_x.append(est_x[-1] + (real_x[i]-real_x[i-1])
                 + np.random.normal(0, RUIDO_SIGMA))
    est_y.append(est_y[-1] + (real_y[i]-real_y[i-1])
                 + np.random.normal(0, RUIDO_SIGMA))

error = np.sqrt((real_x[-1]-est_x[-1])**2 + (real_y[-1]-est_y[-1])**2)
print(f"Error acumulado: {error:.4f} m")
if error > 0.3:
    print(f"[REGLA SE] Error ({error:.2f}m) > umbral (0.3m)")
    print("           -> Accion: solicitar_relocalizacion_lidar()")

fig, ax = plt.subplots(figsize=(7, 7))
ax.plot(real_x, real_y, "b-", linewidth=2, label="Trayectoria real")
ax.plot(est_x,  est_y,  "r--", linewidth=1.5, label="Odometria estimada")
ax.plot(est_x[-1], est_y[-1], "rx", markersize=12, label="Posicion estimada
    final")
ax.plot(real_x[-1], real_y[-1], "b*", markersize=12, label="Posicion real final"
    )
ax.set_aspect("equal"); ax.grid(True, alpha=0.3); ax.legend()
ax.set_title(f"Deriva de Odometria | Error: {error:.4f} m")
plt.tight_layout(); plt.show()
```

Los laboratorios de los capítulos anteriores han puesto de manifiesto dos tipos de incertidumbre que el robot debe gestionar: la **incertidumbre sensorial** (los sensores fallan, acumulan deriva, devuelven ruido) y la **incertidumbre de estado** (la batería se agota, la temperatura sube). Ambas se gestionan con los mecanismos que ya conocemos: fusión sensorial, filtros estadísticos y reglas del SE.

Pero existe un tercer tipo de incertidumbre que ninguna de esas herramientas resuelve por sí sola: la **incertidumbre del lenguaje humano**. Cuando un operario dice "lleva esto a la zona de montaje", el robot no puede resolver esa orden con un sensor ni con una regla booleana. Necesita un modelo que entienda el significado de la frase, lo descomponga en pasos concretos y lo traduzca a los *Hechos* que el SE sí sabe procesar. Esa es exactamente la función de los Modelos de Lenguaje de Gran Escala (LLMs) en la robótica moderna.

4.3. Tendencias Actuales: Sistemas Expertos Híbridos

Los **LLMs** han demostrado una capacidad extraordinaria para comprender instrucciones en lenguaje natural. Sin embargo, un LLM por sí solo no puede garantizar seguridad: puede "alucinar" y no puede verificar reglas de seguridad con certeza absoluta. La solución son los **Sistemas Híbridos**: el LLM actúa como intérprete de lenguaje natural, y el SE actúa como garante de seguridad. El LLM **no ejecuta directamente**: solo propone. El SE aprueba o rechaza.

4.3.1. Laboratorio 4.2: Interfaz LLM-Experto con API real

Entorno: Conda. El script funciona en modo simulado sin necesidad de ninguna clave de API.

```
conda activate robots-expertos-mia
python labs/lab_4/lab_4_2_interfaz_llm_experto.py

# Para usar la API real de Anthropic (opcional):
export ANTHROPIC_API_KEY="sk-ant-..."
python labs/lab_4/lab_4_2_interfaz_llm_experto.py
```

```python
# Laboratorio 4.2 - Interfaz LLM-Experto
import json

def interprete_con_api_anthropic(comando_voz, api_key):
    import anthropic
    client = anthropic.Anthropic(api_key=api_key)
    system_prompt = """Eres el modulo de interpretacion de un robot industrial.
Devuelve UNICAMENTE JSON con: emergencia(bool), causa(str|null),
tarea("navegacion"|"recarga"|"inspeccion"|"espera"|null),
destino(str|null), prioridad("alta"|"media"|"baja")."""
    msg = client.messages.create(
        model="claude-haiku-4-5-20251001", max_tokens=200,
        system=system_prompt,
        messages=[{"role": "user", "content": comando_voz}])
    try: return json.loads(msg.content[0].text.strip())
    except json.JSONDecodeError: return {}

def motor_experto_seguridad(hechos):
    if not hechos: print("[SE] JSON invalido."); return
    if hechos.get("emergencia"):
        print(f"[SE] PRIORIDAD MAXIMA. Causa: {hechos.get('causa','desconocida')
            }")
        print("[SE] Ejecutando protocolo de evacuacion.")
    elif hechos.get("tarea") == "navegacion":
        print(f"[SE] Navegacion -> '{hechos.get('destino')}' | {hechos.get('
            prioridad')}")
    elif hechos.get("tarea") == "recarga":
        print("[SE] Navegando a estacion de carga.")
    elif hechos.get("tarea") == "inspeccion":
        print(f"[SE] Inspeccion en '{hechos.get('destino')}'. Activando sensores
            .")
    else: print(f"[SE] {hechos}. Sin accion definida.")

def interprete_simulado(cmd):
    c = cmd.lower()
    if any(w in c for w in ["fuego", "humo", "emergencia"]):
        return {"emergencia": True, "causa": "posible_incendio",
                "tarea": None, "destino": None, "prioridad": "alta"}
    if any(w in c for w in ["ve a", "navega a", "muevete a"]):
        dest = c
        for p in ["muevete a ", "navega a ", "ve a "]:
```

```
            if p in dest: dest = dest.split(p, 1)[1].strip(); break
        return {"emergencia": False, "causa": None, "tarea": "navegacion",
                "destino": dest.rstrip(".,!?"), "prioridad": "media"}
    if any(w in c for w in ["recarga", "bateria", "carga"]):
        return {"emergencia": False, "causa": None, "tarea": "recarga",
                "destino": "base_carga", "prioridad": "alta"}
    if "inspeccion" in c or "revisa" in c:
        return {"emergencia": False, "causa": None, "tarea": "inspeccion",
                "destino": "zona_B", "prioridad": "baja"}
    return {}

for cmd in ["Cuidado, hay humo!", "ve a la zona B",
            "necesito que te recargues", "revisa la zona de montaje"]:
    print("-" * 55)
    hechos = interprete_simulado(cmd)
    print(f"[LLM-sim] '{cmd}'")
    print(f"[LLM->SE] {json.dumps(hechos, ensure_ascii=False)}")
    motor_experto_seguridad(hechos)
```

4.4. Práctica Final: El Robot que Razona

El proyecto final integra todos los contenidos de la unidad en un único sistema cohesionado. El alumno debe diseñar e implementar un **agente robótico autónomo completo**: percibe, razona con un SE y actúa de forma segura y justificable.

4.4.1. Arquitectura del Sistema Integrado

Todos los grupos deben orquestar un grafo de ROS 2 con **mínimo tres nodos propios**, siguiendo el patrón de integración de la Sección 4.1:

1. **Nodo de Percepción/Entrada:** Lee sensores o LLM y publica hechos simbólicos.
2. **Nodo del SE (Cerebro):** Recibe hechos, infiere el modo de operación y publica la decisión con justificación.
3. **Nodo de Actuación:** Traduce la decisión en `cmd_vel`.

El repositorio incluye `labs/proyecto_final/orquestador_final.py` como punto de partida intencionadamente vacío. Cada grupo construye su propia solución desde cero.

4.4.2. Propuesta 1 — Logística con Prioridad de Seguridad

Objetivo: Robot móvil que transporta material entre zona A y zona B, gestionando obstáculos, batería baja y sobretemperatura. El SE debe demostrar política de prioridad explícita: seguridad siempre por encima de la misión.

Arquitectura recomendada:

```
nodo_percepcion.py    # suscribe /scan -> publica /estado_entorno (String)
nodo_telemetria.py    # simula bateria+temp -> publica /telemetria (String JSON)
nodo_cerebro_se.py    # suscribe ambos -> publica /modo_operacion (String)
nodo_actuacion.py     # suscribe /modo_operacion -> publica /cmd_vel
```

Hoja de ruta:

1. **Semana 1 — Infraestructura de tópicos:** Arranca el simulador y verifica con `ros2 topic list` que `/scan` ya está activo. Crea `nodo_percepcion.py` usando la plantilla de la Sección 2.3: suscríbete a `/scan` y publica en `/estado_entorno` un mensaje `String` con `"LIBRE"` u `"OBSTACULO"` según la distancia mínima frontal. Verifica con `ros2 topic echo /estado_entorno`.

2. **Semana 1 — Telemetría simulada:** Crea `nodo_telemetria.py` con un timer a 1 Hz. Publica JSON con `bateria` (empieza en 100, decrece 0.5 por tick) y `temperatura` (`random.gauss(70, 8)`).

3. **Semana 2 — Motor de reglas:** En `nodo_cerebro_se.py` reutiliza `SistemaExpertoSalud` del Lab 3.5 siguiendo el patrón de la Sección 4.1. Añade al menos: `IF bateria < 15 THEN modo = RECARGA`, `IF obstaculo AND modo==TRANSPORTE THEN modo = EVASION`, `IF temperatura > 85 THEN modo = PARADA_TERMICA`.

4. **Semana 2 — Prioridad y actuación:** En `nodo_actuacion.py` implementa la tabla de prioridades: `PARADA_TERMICA` bloquea cualquier movimiento, `RECARGA` prevalece sobre `TRANSPORTE`. Usa el controlador difuso del Lab 3.4 para regular la velocidad.

5. **Semana 3 — Validación forzada:** Añade un parámetro `-forzar-bateria-baja` a `nodo_telemetria.py`. Documenta con capturas que el SE cambia el modo a `RECARGA` y el robot deja de avanzar.

Qué investigar:

- *Parámetros de ROS 2:* `declare_parameter` y `get_parameter` para hacer configurables los umbrales del SE sin tocar el código.
- *Mensajes personalizados:* busca *"ROS 2 custom message type tutorial"* para crear un tipo `EstadoRobot.msg` en lugar de usar `String` con JSON.
- *rqt_plot:* herramienta gráfica para visualizar la evolución temporal de los tópicos. Imprescindible para el entregable.

4.4.3. Propuesta 2 — Inspección Autónoma de Planta

Objetivo: Recorrido por una lista de waypoints con cambio automático a modo retorno ante fallo de sensor, pérdida de comunicación o deriva de odometría excesiva.

Arquitectura recomendada:

```
nodo_planificador.py  # gestiona la lista de waypoints y el punto actual
nodo_monitor_se.py    # detecta anomalias y decide si continuar o abortar
nodo_odometria.py     # suscribe /odom, detecta deriva (Lab 4.1 integrado)
nodo_actuacion.py     # traduce waypoint actual en cmd_vel
```

Hoja de ruta:

1. **Semana 1 — Lista de waypoints:** Define los puntos de inspección como una lista de tuplas `(x, y)`. En `nodo_planificador.py`, publica el waypoint actual como `Point` (de `geometry_msgs.msg`). Cuando el robot se acerque a menos de 0.3 m (comparando con `/odom`), avanza al siguiente.
2. **Semana 1 — Control hacia el objetivo:** En `nodo_actuacion.py`, calcula el ángulo y distancia hacia el waypoint con `math.atan2`. Implementa un controlador proporcional simple: `angular.z = Kp * error_angulo`.
3. **Semana 2 — Detección de deriva:** Integra el código del Lab 4.1 en `nodo_odometria.py`. Publica el error estimado en `/deriva_acumulada`. El SE activa retorno cuando supere 0.5 m.
4. **Semana 2 — Fallo de sensor por timeout:** Añade en `nodo_monitor_se.py` una regla que detecte ausencia de mensajes en `/scan` durante más de 2 segundos usando `self.get_clock().now().nanoseconds`.
5. **Semana 3 — Modo retorno:** Cuando el SE activa `MODO_RETORNO`, el planificador revierte la lista de waypoints y navega de vuelta al origen.

Qué investigar:

- *Mensaje Odometry:* Ejecuta `ros2 interface show nav_msgs/msg/Odometry`. La posición está en `pose.pose.position` y la orientación en quaterniones. Investiga `tf_transformations` para convertir quaternión a ángulo de Euler.
- *Máquinas de estado:* Busca *"SMACH ROS 2"* o implementa una máquina de estados con un diccionario Python para gestionar `INSPECCIONANDO`, `ESPERANDO`, `RETORNANDO` y `ABORTADO`.
- *Visualización de trayectoria:* Investiga el tipo de mensaje `Path` de `nav_msgs.msg` para visualizar la trayectoria planificada vs real en RViz2.

4.4.4. Propuesta 3 — Asistente Colaborativo con Lenguaje Natural

Objetivo: Recepción de órdenes en lenguaje natural, extracción de Hechos mediante LLM y validación por el SE antes de actuar. El SE actúa como garante: el LLM propone, el SE aprueba o rechaza.

Arquitectura recomendada:

```
nodo_interfaz_llm.py  # lee input del usuario -> llama LLM -> publica /
    hechos_llm
nodo_se_validador.py  # valida los Hechos con reglas -> publica /orden_validada
nodo_ejecutor.py      # traduce la orden validada en cmd_vel
nodo_justificador.py  # suscribe /orden_validada -> imprime explicacion en texto
```

Hoja de ruta:

1. **Semana 1 — Interfaz simulada:** Empieza *sin* API real. Reutiliza `interprete_simulado()` del Lab 4.2 en `nodo_interfaz_llm.py`. Lee `input()` con un timer y publica el JSON resultante como `String` en `/hechos_llm`. Verifica con `ros2 topic echo /hechos_llm`.

2. **Semana 1 — Motor de validación:** En `nodo_se_validador.py` implementa al menos estas reglas que pueden *bloquear* una orden del LLM: `IF tarea==navegacion AND bateria<20 THEN rechazar`, `IF emergencia==True THEN rechazar_todo_y_evacuar`, `IF destino_desconocido THEN pedir_confirmacion`.

3. **Semana 2 — Integración API real (recomendado):** Sustituye `interprete_simulado()` por la llamada real a `claude-haiku`. Prueba con 10 comandos distintos y documenta qué Hechos extrae correctamente el modelo.

4. **Semana 2 — Justificador conversacional:** El `nodo_justificador.py` debe formular frases en lenguaje natural. Por ejemplo: *"No puedo ir a la zona B porque la batería está al 18 %, por debajo del umbral del 20 %."* Usa f-strings con los campos del JSON.

5. **Semana 3 — Prueba de robustez:** Diseña 5 órdenes que el SE debe rechazar y 5 que debe aprobar. Documenta el comportamiento real con capturas de consola.

Qué investigar:

- *Prompting estructurado:* Investiga *"structured output JSON mode LLM"* y cómo manejar `json.JSONDecodeError` cuando el modelo no respeta el formato.
- *Ollama en local:* Si el centro dispone de hardware adecuado, investiga cómo ejecutar `llama3:8b` en local y compara la latencia con la API de Anthropic.
- *Historial de conversación:* El LLM no tiene memoria entre llamadas. Investiga cómo mantener una lista de `dicts` `{"role": ..., "content": ...}` para responder a preguntas de seguimiento.

4.4.5. Propuesta 4 — Mantenimiento Predictivo Básico

Objetivo: Monitorización de telemetría en tiempo real, clasificación por niveles de alerta (informativo, mantenimiento, crítico) y generación de un log estructurado para análisis post-sesión.

Arquitectura recomendada:

```
nodo_generador_telemetria.py  # simula sensores con degradacion progresiva
nodo_se_diagnostico.py        # clasifica alertas con el motor de reglas
nodo_controlador_difuso.py    # ajusta velocidad segun nivel de alerta
nodo_logger.py                # guarda alertas en CSV con timestamp
```

Hoja de ruta:

1. **Semana 1 — Telemetría realista:** En `nodo_generador_telemetria.py`, simula cuatro sensores con comportamiento dinámico: temperatura crece lentamente mientras el robot se mueve (`temp += 0.1*dt`) y se enfría en parada; el voltaje decrece de forma no lineal, rápido por debajo del 30 %. Publica JSON a 2 Hz.
2. **Semana 1 — Niveles de alerta:** Amplía `SistemaExpertoSalud` del Lab 3.5 con niveles numéricos: 0 = nominal, 1 = informativo, 2 = mantenimiento, 3 = crítico. El nodo publica el nivel máximo activo en `/nivel_alerta` como `Int32`.
3. **Semana 2 — Controlador difuso de potencia:** En `nodo_controlador_difuso.py` reutiliza el Lab 3.4 pero usa `nivel_alerta` (0–3) como entrada en lugar de distancia: nivel 3 → 0 % velocidad, nivel 0 → 100 %.
4. **Semana 2 — Logger estructurado:** En `nodo_logger.py`, suscríbete a `/telemetria` y `/nivel_alerta`. Cada vez que el nivel cambia, escribe una línea en CSV con: timestamp, nivel_anterior, nivel_nuevo, telemetria completa, regla activada. Usa `csv.DictWriter` de la librería estándar.
5. **Semana 3 — Análisis post-sesión:** Con los datos del CSV, crea un script Python separado (fuera de ROS 2) que use `matplotlib` para representar la evolución temporal de temperatura, voltaje y nivel de alerta en la misma figura con dos ejes Y.

Qué investigar:

- *Ventana deslizante (sliding window):* En lugar de activar una alerta ante un único valor anómalo, actívala solo si la media de los últimos N valores supera el umbral. Implementa con `collections.deque(maxlen=N)`.
- *ros2 bag record:* Investiga cómo grabar todos los tópicos activos en un fichero para reproducirlos después (`ros2 bag play`). Es el estándar industrial para registrar sesiones de operación.
- *Detección estadística de anomalías:* Busca *"Z-score anomaly detection Python"* para detectar valores que se desvíen más de 2 desviaciones típicas de la media histórica, en lugar de umbrales fijos.

4.4.6. Evaluación del Proyecto

Arquitectura propia: Mínimo 3 nodos ROS 2 propios con tópicos bien definidos y documentados.

Navegación funcional: El robot alcanza el objetivo sin colisiones en Gazebo o PyBullet (ver Plan B abajo).

Razonamiento demostrable: Se fuerza al menos un escenario de fallo y se documenta cómo el SE toma el control y cambia la misión en tiempo real.

Trazabilidad: El robot imprime el razonamiento completo. Ejemplo mínimo: `[SE] Cambiando mision a RECARGA porque BATERIA < 15%`.

Controlador difuso: Al menos un parámetro de actuación usa el controlador del Lab 3.4 en lugar de un umbral binario.

Memoria técnica: Documento de 4–6 páginas con diagrama del grafo de nodos, tabla de reglas del SE, resultados de al menos tres experimentos forzados y reflexión sobre las limitaciones encontradas.

Plan B: entorno sin Gazebo

Si Gazebo no funciona correctamente en algún equipo (problemas de drivers gráficos o WSL 2), el proyecto puede validarse íntegramente con **PyBullet** y un generador de datos de sensor sintético similar al del Lab 4.1. Todos los nodos ROS 2 funcionan igual; solo cambia el origen de los datos del sensor. Comunicad esta situación al profesor antes de la Semana 2 para acordar la configuración alternativa.

4.4.7. Conclusión del Bloque

Con la finalización de este proyecto se habrán integrado los cuatro pilares abordados en la unidad: la **Mecánica y el Hardware** (Cap. 1), el **Middleware de comunicaciones distribuidas** con ROS 2 (Cap. 2), la **IA Simbólica y el Control Inteligente** incluyendo lógica difusa (Cap. 3), y la **integración con LLMs** para la interacción en lenguaje natural (Cap. 4).

El robot construido en este proyecto no es solo un actuador en el mundo físico: es también un **generador masivo y continuo de datos estructurados**. Cada segundo de operación produce logs de telemetría, trayectorias cartesianas, decisiones del Sistema Experto con su justificación, lecturas de sensores y comandos de velocidad. Todos esos datos están listos para alimentar *pipelines* de análisis más amplios: desde dashboards de monitorización de flotas hasta modelos de mantenimiento predictivo entrenados con series temporales de sensores. Este es precisamente el puente natural con el resto del Curso de Especialización en IA y Big Data.